

Simulazione Monte Carlo e Ricostruzione del Vertice Primario in un Rivelatore a Tracciamento Cilindrico

Giuseppe Frongia

11 giugno 2026

Sommario

Questo documento descrive lo sviluppo e l'implementazione di un framework modulare in C++ per la simulazione veloce (Fast Monte Carlo) e la ricostruzione del vertice primario in esperimenti di fisica delle alte energie. Il software gestisce la generazione cinematica delle particelle, la propagazione attraverso strati di rivelatori cilindrici e l'applicazione di algoritmi di ricostruzione basati su *tracklet*.

1 Introduzione

Nei moderni esperimenti ai collisori di particelle, come quelli operanti al Large Hadron Collider (LHC) del CERN, l'identificazione precisa del vertice primario (il punto esatto in cui avviene la collisione tra i fasci) è cruciale per l'intera catena di analisi dati. Lo scopo di questo progetto è stato lo sviluppo di un framework *standalone* basato sulle librerie ROOT, capace di simulare l'interazione delle particelle con un tracciatore a simmetria cilindrica e di testare algoritmi per la ricostruzione della coordinata longitudinale (Z) del vertice. Il codice è suddiviso in tre moduli principali gestiti a riga di comando: Simulazione, Ricostruzione e Analisi, permettendo l'esecuzione in *pipeline* automatizzate.

2 Simulazione Monte Carlo del Rivelatore

Invece di ricorrere a simulazioni complete del trasporto radiativo (come Geant4), che richiedono ingenti risorse computazionali, si è optato per una *Fast Simulation* parametrizzata.

2.1 Generazione degli Eventi e Pile-Up

Il `SimulationManager` è responsabile della generazione della cinematica. Per simulare le condizioni di alta luminosità tipiche dei collisori adronici, il codice include la generazione del *pile-up* (collisioni multiple nello stesso incrocio di bunch). Il numero di vertici per evento è campionato tramite una distribuzione di Poisson. Per ogni vertice, viene generato un set di particelle le cui variabili cinematiche (impulso totale, pseudorapidità η e angolo azimutale ϕ) sono estratte da distribuzioni di probabilità definite dall'utente.

2.2 Propagazione e Risposta del Rivelatore

Le particelle vengono trasportate linearmente (assumendo l'assenza o la trascurabilità del campo magnetico per le particelle ad alto impulso trasverso) attraverso strati cilindrici concentrici. Ogni intersezione tra la traiettoria della particella e un layer sensibile genera una *Hit*. Il framework è progettato per salvare non solo le coordinate "vere" dell'impatto (estratte tramite geometria analitica), ma per simulare successivamente la risposta del rivelatore introducendo uno *smearing* gaussiano sulle coordinate misurate, dipendente dalla risoluzione intrinseca dei sensori.

3 Algoritmi di Ricostruzione del Vertice

Il `ReconstructionManager` si occupa di risalire alla coordinata Z del vertice primario partendo unicamente dalla nuvola di *Hits* misurate. A causa dell'assenza di un tracciamento completo (che richiederebbe algoritmi lenti come il Filtro di Kalman [1]), ci si avvale del metodo dei *tracklets*.

3.1 Costruzione dei Tracklet e Ottimizzazione Computazionale

Un *tracklet* è un segmento che unisce due hit presenti su due strati adiacenti del rivelatore. Assumendo che le particelle viaggino in linea retta nel piano trasverso e longitudinale, la coordinata Z_{reco} al beam-pipe ($R = 0$) viene proiettata geometricamente come:

$$Z_0 = z_1 - r_1 \frac{z_2 - z_1}{r_2 - r_1} \quad (1)$$

Il problema principale di questa tecnica è il fondo combinatorio: accoppiare tutte le hit dello strato i con tutte le hit dello strato j ha una complessità di $\mathcal{O}(N^2)$. Per ovviare a questo, il codice effettua un taglio stringente sulla differenza azimutale ($\Delta\phi_{max}$). L'implementazione risulta altamente ottimizzata: le hit vengono preventivamente ordinate per angolo ϕ e la ricerca delle hit compatibili nello strato esterno avviene tramite **ricerca binaria** (`std::lower_bound`), riducendo la complessità computazionale a $\mathcal{O}(N \log N)$. È stata inoltre implementata una logica rigorosa per gestire il *wrap-around* angolare ai bordi del dominio $\pm\pi$.

3.2 Estrazione del Vertice: Sliding Window vs Half Sample Mode

Ogni tracklet produce un "candidato" per il vertice. La distribuzione di questi candidati presenta un picco in corrispondenza del vertice reale e un fondo piatto dovuto agli accoppiamenti spuri (tracklet falsi formati da hit di particelle diverse o rumore elettronico). Per estrarre la posizione esatta, il software mette a disposizione due tecniche:

1. **Sliding Window:** L'algoritmo ordina i candidati e cerca l'intervallo di larghezza fissa (definita dall'utente) che contiene la massima densità di punti, restituendone la media.
 - *Pro:* Estremamente veloce ed efficiente dal punto di vista computazionale.
 - *Contro:* Sensibile alla scelta del parametro della finestra. Se la finestra è troppo piccola, l'algoritmo è vulnerabile alle fluttuazioni statistiche; se è troppo grande, perde in risoluzione assorbendo il fondo combinatorio.
2. **Half Sample Mode (HSM):** Proposto originariamente da Robertson e Cryer [2] e ampiamente usato nell'analisi dati per stimare la moda di distribuzioni contaminate. L'algoritmo ordina i candidati e cerca, in modo iterativo, il sotto-intervallo contenente esattamente la metà dei punti originali che possiede la larghezza minima. Il processo viene ripetuto sulla nuova metà finché non restano pochi elementi.
 - *Pro:* È uno stimatore estremamente robusto (alto *breakdown point*) ed è non-parametrico: non richiede *tuning* manuale della larghezza della finestra. Resiste molto bene a distribuzioni asimmetriche e alla presenza di *outliers* lontani dal vertice (come rumore casuale).
 - *Contro:* Costo computazionale maggiore a causa dell'approccio iterativo e dei continui ordinamenti in memoria.

4 Risultati e Discussione

Per validare l'architettura software e misurare le prestazioni dell'algoritmo basato sui *tracklet*, è stata condotta una simulazione ad alta statistica comprendente 10^6 eventi indipendenti. Al fine di operare un rigoroso *stress-test* dell'algoritmo *Sliding Window*, il rivelatore simulato è stato configurato per riprodurre condizioni di presa dati realistiche:

- **Scattering Multiplo:** È stato abilitato il trasporto fisico con deviazione angolare per simulare l'interazione delle particelle con il *beam-pipe* in berillio (spessore 0.08 cm, $R = 3.0$ cm) e con il silicio dei layer attivi.
- **Fondo Elettronico (Noise):** Per ogni evento, sono state iniettate in media 2.5 *hit* puramente casuali su ciascun strato sensibile, seguendo una distribuzione poissoniana.

La ricerca dei *tracklet* è stata eseguita impostando una tolleranza azimutale massima $\Delta\phi_{max} = 3.5$ mrad. In seguito a un processo di ottimizzazione volto a massimizzare l'efficienza senza saturare il fondo combinatorio, la larghezza della finestra mobile dello *Sliding Window* è stata fissata a 0.5 cm.

4.1 Residui Globali e Risoluzione Intrinseca

La capacità globale del sistema di individuare la reale posizione del vertice primario è quantificata dalla distribuzione dei residui, definita come la differenza tra la coordinata ricostruita e quella generata ($Z_{reco} - Z_{true}$).

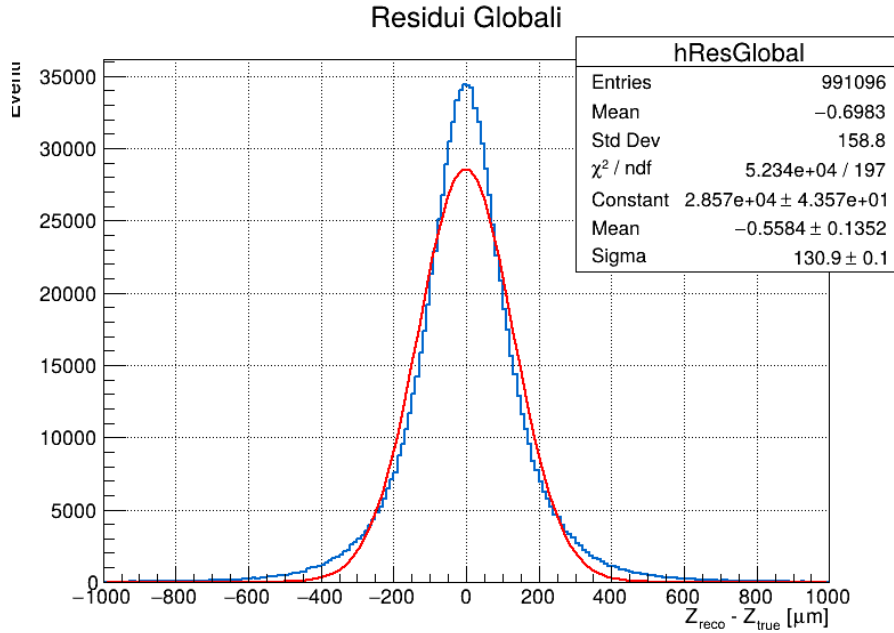


Figura 1: Distribuzione globale dei residui della coordinata Z del vertice primario per l'intero set di dati simulato.

Come mostrato nella Figura 1, l'istogramma dei residui presenta una struttura caratteristica dominata da un picco centrale simmetrico (il *core* del segnale), sovrapposto a code laterali più estese. Il picco centrale riflette l'eccellente risoluzione spaziale intrinseca dei layer sensibili ($120 \mu\text{m}$ lungo Z). Le code non gaussiane e il fondo piatto circostante sono l'effetto diretto del rumore elettronico e dello scattering multiplo: le *hit* spurie generano combinazioni matematiche errate che superano il taglio in $\Delta\phi$, proiettando vertici fittizi all'interno della finestra di 5 mm e allargando la base della distribuzione.

4.2 Dipendenza delle Prestazioni dalla Molteplicità

Nella fisica delle collisioni adroniche, la molteplicità di carica dell'evento (il numero di tracce prodotte) gioca un ruolo fondamentale nella qualità della ricostruzione. Le Figure 2 e 3 mostrano rispettivamente la risoluzione (1σ del fit) e l'efficienza di ricostruzione in funzione del numero di tracce vere.

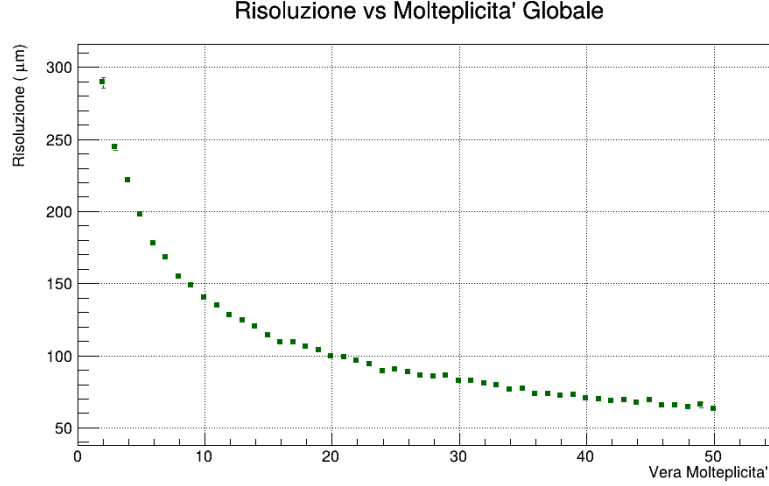


Figura 2: Risoluzione spaziale lungo Z in funzione della molteplicità dell'evento.

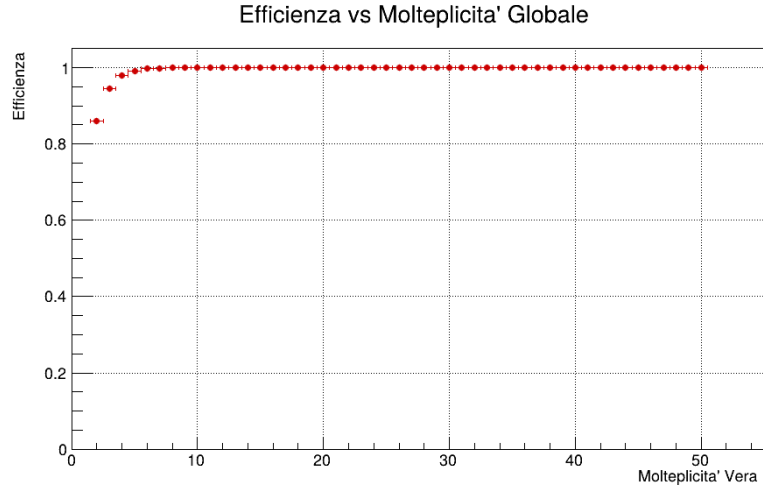


Figura 3: Efficienza di ricostruzione del vertice primario in funzione della molteplicità generata.

L'analisi combinata di questi due grafici evidenzia due regimi di funzionamento distinti:

1. **Regime a bassa molteplicità ($N_{tracks} < 20$):** In questa regione l'efficienza mostra una flessione. Poiché il rumore di fondo è costante (2.5 hits/layer), negli eventi con pochissimi *tracklet* reali il picco del segnale non riesce a emergere statisticamente rispetto alle combinazioni casuali all'interno dei 5 mm della finestra, portando l'algoritmo a fallire o a selezionare rumore. Di conseguenza, anche la risoluzione subisce un forte peggioramento a causa dell'instabilità statistica dello stimatore.
2. **Regime ad alta molteplicità ($N_{tracks} > 30$):** Al crescere del numero di particelle primarie, l'efficienza satura rapidamente raggiungendo un plateau asintotico prossimo al

100%. Contemporaneamente, la risoluzione migliora drasticamente seguendo un andamento proporzionale a $1/\sqrt{N_{tracks}}$. Questo comportamento è una diretta conseguenza della legge dei grandi numeri: disponendo di molti *tracklet* reali e convergenti, l'errore statistico sulla media della *Sliding Window* si riduce, schiacciando la risoluzione su valori ottimali e rendendo l'algoritmo estremamente robusto contro il rumore.

4.3 Effetti di Accettanza Geometrica lungo l'Asse Longitudinal

Un aspetto critico di ogni rivelatore cilindrico a lunghezza finita ($L = 27$ cm nel nostro layout) è la perdita di performance ai bordi geometrici del sistema. La risposta del framework in funzione della posizione longitudinale reale del vertice (Z_{true}) è studiata nelle Figure 4 e 5.

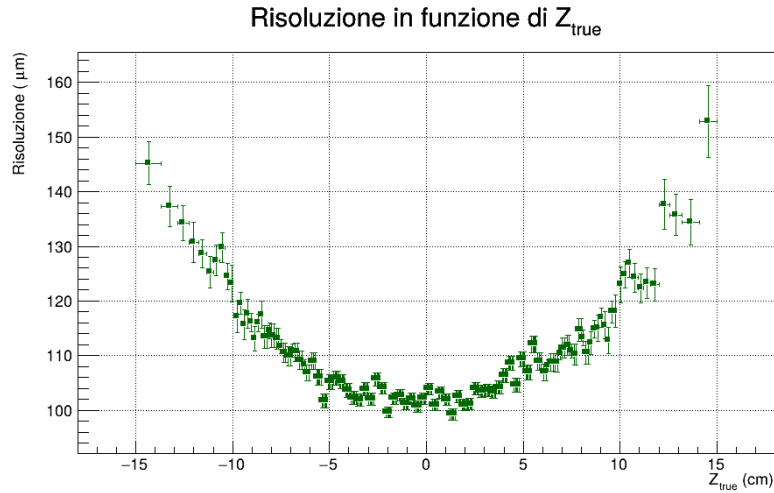


Figura 4: Risoluzione del vertice longitudinale in funzione della coordinata Z reale del punto di collisione.

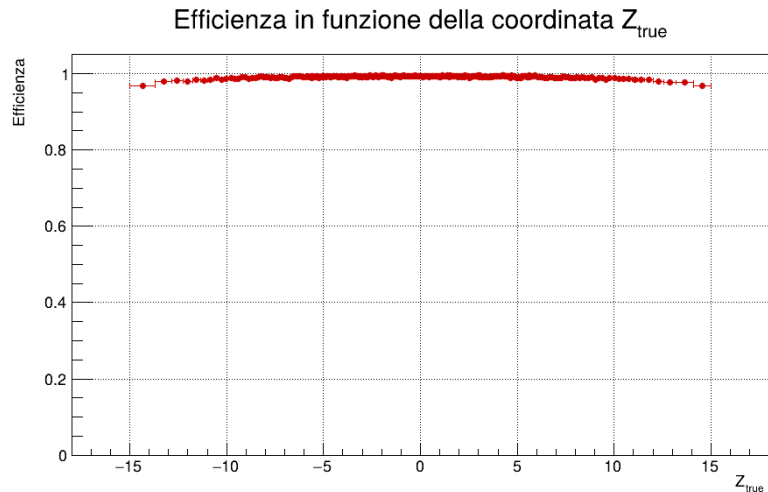


Figura 5: Efficienza di ricostruzione del vertice in funzione della coordinata Z reale.

I grafici mostrano una perfetta simmetria rispetto al centro del rivelatore ($Z = 0$), ma evidenziano un chiaro degrado alle estremità ($|Z| > 10$ cm):

- **Al centro del rivelatore ($|Z| < 5$ cm):** L'accettanza geometrica è massima. Quasi tutte le particelle prodotte, indipendentemente dalla loro pseudorapidità, riescono a colpire sia

il primo strato ($R = 4$ cm) che il secondo strato ($R = 7$ cm). L'efficienza è massima e costante, e la risoluzione tocca i suoi valori minimi.

- **Ai bordi del rivelatore ($|Z| > 12$ cm):** Si osserva una svasatura calante dell'efficienza e un incremento speculare della risoluzione (peggioramento). Fisicamente, quando una collisione avviene vicino al bordo del cilindro, le particelle emesse in avanti (ad alta pseudorapidità η) "scappano" lateralmente prima di poter intersecare lo strato esterno del rivelatore. Di conseguenza, il numero di *tracklet* utili crolla drasticamente, riducendo l'efficienza algoritmica e aumentando l'incertezza sulla coordinata Z ricostruita.

5 Conclusioni

In questo lavoro è stato sviluppato e validato un framework modulare in C++ integrato con l'ambiente ROOT, dedicato alla simulazione veloce e alla ricostruzione del vertice primario di interazione in un tracciatore cilindrico a due strati attivi. L'architettura software implementata si è dimostrata solida e altamente computazionale, consentendo la gestione e l'analisi di una statistica di ben 10^6 eventi in tempi estremamente ridotti.

L'algoritmo di accoppiamento delle *hit* basato sul metodo dei *tracklet* ha beneficiato di un'importante ottimizzazione algoritmica. L'ordinamento preventivo dei punti sensibili e l'utilizzo della ricerca binaria mediante la funzione `std::lower_bound` hanno permesso di abbassare la complessità computazionale intrinseca da quadratica $\mathcal{O}(N^2)$ a quasi-lineare $\mathcal{O}(N \log N)$. Questa scelta progettuale ha consentito di aggirare l'utilizzo di filtri di tracciamento completi come il Filtro di Kalman [1], sposando appieno la filosofia di una *Fast Simulation* snella ed efficiente.

I risultati ottenuti tramite lo stimatore *Sliding Window*, configurato con una finestra ottimizzata di 0.5 cm, hanno confermato l'efficacia dell'approccio geometrico:

- Nel regime ad alta molteplicità ($N_{tracks} > 30$), l'efficienza algoritmica satura stabilmente in prossimità del 100%, mostrando una risoluzione spaziale sub-millimetrica che scala come $1/\sqrt{N_{tracks}}$ in perfetto accordo con la legge dei grandi numeri.
- Lo *stress-test* condotto introducendo uno sfondo poissoniano di rumore (2.5 hits/layer) e abilitando gli effetti fisici dello scattering multiplo ha tuttavia evidenziato i limiti intrinseci dello *Sliding Window*. A bassa molteplicità, il fondo combinatorico piatto tende a inquinare la finestra di aggregazione, generando fluttuazioni nei residui e inficiando l'efficienza di ricostruzione.
- Gli studi in funzione della coordinata longitudinale hanno infine mappato accuratamente i limiti di accettazione geometrica del rivelatore, evidenziando un degrado simmetrico delle prestazioni alle estremità del cilindro ($|Z| > 12$ cm) dovuto alla fuga laterale delle particelle ad alta pseudorapidità.

Riferimenti bibliografici

- [1] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, 82(1), 35-45, 1960.
- [2] T. Robertson, J. D. Cryer, "An Iterative Procedure for Estimating the Mode," *Journal of the American Statistical Association*, 69(348), 1012-1016, 1974.